

# Lab 02

Phung Cao Thi Ngoc

2025.12.08

# Contents

|                                          |   |
|------------------------------------------|---|
| 1. Introduction Libraries .....          | 3 |
| 2. Breadth-First Search.....             | 3 |
| 2.1 Breadth-First Search Algorithm ..... | 3 |
| 2.2 Input of BFS .....                   | 4 |
| 2.3 Output of BFS .....                  | 4 |
| 3. Depth-First Search (Exercise 1) ..... | 4 |
| 3.1 Depth-First Search Algorithm .....   | 4 |
| 3.2 Input of DFS .....                   | 5 |
| 3.3 Output of DFS .....                  | 5 |
| 4. A* (Exercise 2) .....                 | 5 |
| 4.1 A* Algorithm .....                   | 5 |
| 4.2 Input of A* .....                    | 6 |
| 4.3 Output of A* .....                   | 6 |
| 5. Compare BFS and A* (Exercise 3).....  | 7 |
| 5.1 Caculate time of BFS and A* .....    | 7 |
| 5.2 Input of BFS and A* .....            | 7 |
| 5.3 Output of BFS and A* .....           | 8 |

# 1. Introduction Libraries

```
from collections import deque
import heapq
from time import perf_counter
```

I import three important libraries. deque is used to make a fast queue for the BFS algorithm. heapq is used to create a priority queue for A\*, so the algorithm can always pick the node with the smallest  $f(n)$ . Finally, perf\_counter is used to measure how long each algorithm takes, so I can compare the performance of BFS and A\*.

## 2. Breadth-First Search

### 2.1 Breadth-First Search Algorithm

```
def bfs_shortest_path(graph, start, goal):
    """
    Finds the shortest path in an unweighted graph using BFS.
    """
    queue = deque([[start]]) # Queue storing paths, start with ['A']
    visited = set() # Set to track visited nodes

    while queue:
        path = queue.popleft() # Take the first path from the queue
        node = path[-1] # Get the last node in the current path
        if node in visited:
            continue # Skip if node was already visited
        visited.add(node) # Mark node as visited
        if node == goal:
            return path # If node is the goal → return path
        # Explore neighbors
        for neighbor in graph.get(node, []):
            new_path = list(path)
            new_path.append(neighbor)
            queue.append(new_path) # Create a new path and add to queue

    return None # No path found
```

This function uses the Breadth-First Search algorithm to find the shortest path in an unweighted graph. First, it creates a queue that stores full paths, starting with the path that contains only the start node. It also uses a visited set to make sure each node is processed only once.

Inside the loop, the algorithm takes the first path from the queue and checks the last node in that path. If the node was already visited, it skips it. If it reaches the goal node, it returns the path immediately. Otherwise, it explores all neighbors of the current node, creates new paths, and adds them to the queue.

Because BFS explores the graph level by level, it will always find the shortest path in terms of the number of edges.

## 2.2 Input of BFS

```
# Example graph
graph = {
    "A": ["B", "C"],
    "B": ["D", "E"],
    "C": ["F"],
    "D": [],
    "E": ["F"],
    "F": []
}

start_node = "A"
goal_node = "F"

✓ 0.0s
```

This graph is the input for the BFS algorithm. It is written as an adjacency list, where each node shows the neighbors it can go to. The start\_node is A and the goal\_node is F, so the algorithm will search for the shortest path from A to F.

## 2.3 Output of BFS

```
print(f"Shortest path from {start_node} to {goal_node}")
print(f"{bfs_shortest_path(graph, start_node, goal_node)}")

✓ 0.0s

Shortest path from A to F
['A', 'C', 'F']
```

This part prints the final result of the BFS search. It shows the start and goal nodes, and then displays the shortest path that the BFS algorithm finds in the graph. The output helps confirm that the algorithm works correctly.

## 3. Depth-First Search (Exercise 1)

### 3.1 Depth-First Search Algorithm

```
visited = set() # Set to store visited nodes
traversal_order = [] # To record traversal order
def dfs(node): # Skip if node already visited
    if node in visited:
        return
    visited.add(node) # Mark node as visited
    traversal_order.append(node) # Add to traversal order
    for neighbor in graph[node]: # Recursively visit unvisited neighbors
        dfs(neighbor) # Dive deeper into each branch

✓ 0.0s
```

This code defines the Depth-First Search algorithm. It uses a visited set to make sure each node is only visited once. The algorithm adds each node to traversal\_order and then explores its neighbors by calling the dfs function again. This makes the search go deep into one branch before moving to the next branch.

## 3.2 Input of DFS

```
graph = {
    "A": ["B", "C"],
    "B": ["D", "E"],
    "C": ["F"],
    "D": [],
    "E": ["F"],
    "F": []
}
start_node = "A"
```

This graph is the input for the DFS algorithm. Each node lists the neighbors it can visit. The search starts from node A, and DFS will explore as deep as possible along each branch before backtracking.

## 3.3 Output of DFS

```
dfs(start_node) # Run DFS starting at A
print("DFS Traversal Order:", traversal_order)
```

DFS Traversal Order: ['A', 'B', 'D', 'E', 'F', 'C']

This part runs the DFS function starting from node A and prints the order of the nodes visited. The output shows how DFS goes deep into each branch before moving to the next one. The traversal order confirms that the algorithm works as expected.

## 4. A\* (Exercise 2)

### 4.1 A\* Algorithm

```
def a_star(graph, heuristic, start, goal):
    # Priority queue (min-heap) storing: (f(n), node, path, g(n))
    pq = []

    # Push the start node into the priority queue
    heapq.heappush(pq, (heuristic[start], start, [start], 0))
    # f(start) = g(start)=0 + h(start)

    visited = set() # Tracks visited nodes

    while pq:
        f, node, path, g = heapq.heappop(pq)
        if node in visited: # Skip if already visited
            continue
        visited.add(node)

        if node == goal: # Goal reached → return path
            return path

        for neighbor, cost in graph.get(node, []): # Explore neighbors
            if neighbor not in visited:
                new_g = g + cost # actual cost g(n)
                new_f = new_g + heuristic[neighbor] # f(n) = g(n) + h(n)
                new_path = path + [neighbor]

                # Push the new path into the priority queue
                heapq.heappush(pq, (new_f, neighbor, new_path, new_g))

    return None # No path found
```

This function implements the A\* search algorithm. It uses a priority queue to always choose the next node with the lowest estimated cost, based on the formula  $f(n) = g(n) + h(n)$ . The algorithm keeps track of visited nodes and builds the path step by step. If it reaches the goal node, it returns the complete path. A\* is more informed than BFS because it uses both the actual cost and the heuristic value to guide the search.

## 4.2 Input of A\*

```
# Example Graph (Weighted)
graph = {
    "A": [("B", 1), ("C", 4)],
    "B": [("D", 2), ("E", 5)],
    "C": [("F", 3)],
    "D": [("F", 1)],
    "E": [("F", 2)],
    "F": []
}

# Heuristic values
h = {"A": 7, "B": 6, "C": 2, "D": 1, "E": 2, "F": 0}

start_node = "A"
goal_node = "F"
```

This graph is the input for the A\* algorithm, and each node has neighbors with different edge weights. The heuristic dictionary gives an estimated distance from each node to the goal. The search starts at node A and aims to reach node F. A\* will use both the real costs and the heuristic values to find the best path.

## 4.3 Output of A\*

```
print("Shortest Path using A*:")
print(a_star(graph, h, start_node, goal_node))

✓ 0.0s

Shortest Path using A*:
['A', 'B', 'D', 'F']
```

This part prints the result of the A\* search. It shows the shortest path from the start node to the goal node based on both the edge costs and the heuristic values. The output confirms that the A\* algorithm is working correctly.

## 5. Compare BFS and A\* (Exercise 3)

### 5.1 Caculate time of BFS and A\*

```
def compare_bfs_a_star(graph_for_bfs, graph_for_astar, heuristic, start, goal):  
    # ---- BFS ----  
    t1 = perf_counter()  
    bfs_path = bfs_shortest_path(graph_for_bfs, start, goal)  
    t2 = perf_counter()  
    bfs_time = t2 - t1  
  
    # ---- A* ----  
    t3 = perf_counter()  
    a_star_path = a_star(graph_for_astar, heuristic, start, goal)  
    t4 = perf_counter()  
    a_star_time = t4 - t3  
  
    print("BFS path :", bfs_path)  
    print("BFS time (s) :", bfs_time)  
    print("A* path :", a_star_path)  
    print("A* time (s) :", a_star_time)
```

This function measures how long BFS and A\* take to find a path. It records the start and end time for each algorithm using `perf_counter()`, and then prints both the path and the running time. This helps compare the performance of BFS and A\* on the same graph.

### 5.2 Input of BFS and A\*

```
graph_uniform = {  
    'A': {'B': 1, 'C': 1},  
    'B': {'D': 1},  
    'C': {'D': 1, 'E': 1},  
    'D': {},  
    'E': {'F': 1, 'G': 1},  
    'F': {},  
    'G': {}  
}  
  
heuristic = {  
    'A': 4,  
    'B': 3,  
    'C': 3,  
    'D': 2,  
    'E': 2,  
    'F': 0,  
    'G': 1  
}  
start_node = "A"  
goal_node = "F"  
  
# ----- Convert graph to reuse previous functions -----  
# My BFS uses adjacency list: node -> [neighbors]  
graph_for_bfs = {  
    node: list(neighbors.keys())  
    for node, neighbors in graph_uniform.items()  
}  
  
# My A* uses weighted adjacency list: node -> [(neighbor, cost), ...]  
graph_for_astar = {  
    node: [(nbr, cost) for nbr, cost in neighbors.items()]  
    for node, neighbors in graph_uniform.items()  
}  
✓ 0.0s
```

This section defines the input for both BFS and A\* in the comparison. The `graph_uniform` dictionary shows the graph where all edges have the same weight. The heuristic dictionary gives an estimated cost from each node to the goal.

Because my BFS and A\* functions use different graph formats, I convert the original graph into two versions: `graph_for_bfs` stores only the neighbors of each node, and `graph_for_astar` stores each neighbor with its cost. This allows me to reuse the same BFS and A\* functions without changing their code.

### 5.3 Output of BFS and A\*

```
compare_bfs_astar(graph_for_bfs, graph_for_astar, heuristic, start_node, goal_node)
✓ 0.0s
BFS path : ['A', 'C', 'E', 'F']
BFS time (s) : 9.100000170292333e-06
A* path : ['A', 'C', 'E', 'F']
A* time (s) : 1.1600001016631722e-05
```

In this test, BFS is slightly faster than A\*. Both algorithms find the same path, but the graph is very small, so the heuristic does not give A\* a strong advantage. A\* also has more overhead because it uses a priority queue and calculates  $f(n)$ . Because of this, BFS runs a bit faster in this simple case. However, in larger and more complex graphs, A\* is usually more efficient.